

Deferral/Referral – Lab-Based Practical

5COSC019C Object Oriented Programming

Health Centre Staff Management System

Important Instructions

You are provided with the existing Java-based Health Centre Management System. You must extend the system by applying object-oriented programming principles, collections, GUI programming, exception-safe logic, and unit testing.

This assessment is designed to test your ability to work with an existing codebase, understand its design, modify it safely, and implement realistic industry-style functionality. Marks will not be awarded for hard-coded outputs, incomplete menu options, unused classes, or code that works only for one fixed example.

You must ensure that:

1. The final project compiles and runs without errors.
 2. All new classes are placed inside the correct package.
 3. Existing functionality must not be broken.
 4. You must not use drag-and-drop GUI builders. GUI changes must be coded manually.
 5. You must use appropriate object-oriented design rather than writing all logic in one method.
 6. You must validate user input where required.
 7. You must submit the complete NetBeans project as a zipped folder.
-

TASK 01 – Create a New LaboratoryTechnician Class

15 marks

Create a new Java class called LaboratoryTechnician inside the package HealthCentreCoursework_5COSC019W_Package.

The class must represent laboratory staff who handle diagnostic testing within the health centre.

Your implementation must include the following:

1. Create the LaboratoryTechnician class in the correct package.

2 marks

2. Ensure that LaboratoryTechnician inherits from StaffMember and correctly applies inheritance. The class must reuse the inherited attributes such as name, surname, phone number, and staff ID rather than duplicating them.

3 marks

3. Add the following private instance variables to the class:

- labSection
- certificationLevel
- numberOfTestsHandledPerDay

3 marks

4. Implement a constructor that accepts name and surname as parameters and correctly calls the superclass constructor.

2 marks

5. Implement appropriate setter and getter methods for all new instance variables. The setter for numberOfTestsHandledPerDay must reject negative values by setting the value to 0 if an invalid value is provided.

3 marks

6. Override the toString() method to return a meaningful description of the laboratory technician, including inherited staff details and the new laboratory-specific details.

2 marks

TASK 02 – Implement Safe Staff Status Management

25 marks

The health centre now needs to temporarily deactivate staff members who are unavailable for duty rather than deleting them immediately. This is a common industrial requirement because staff records should be preserved for audit and HR purposes.

You must implement a staff deactivation feature.

1. Modify the StaffMember class by adding a private boolean instance variable called active. New staff members must be active by default.

4 marks

2. Add appropriate getter and setter methods for the active variable.

3 marks

3. In the HealthCentreManager interface, add an abstract method with the following signature:

`void deactivateStaffMember();`

2 marks

4. In WestminsterHealthCentreManager, override the deactivateStaffMember() method. The method must ask the user to enter a staff ID from the keyboard.

3 marks

5. The method must search for the staff member by staff ID. If found, it must set the staff member's status to inactive instead of removing the object from the list.

4 marks

6. If the staff member is found, print the following information:

- staff ID
- name and surname
- staff role
- updated status

4 marks

7. If the staff ID is not found, display a clear error message. The program must not crash.

2 marks

8. Modify the runMenu() method to include this functionality as a new menu option.

3 marks

TASK 03 – Implement Advanced Staff Search and Sorting

25 marks

The current staff list is not suitable for industrial use because it does not allow efficient viewing of staff by status, role, and workload. You must improve the staff listing functionality.

1. Modify StaffMember to implement Comparable<StaffMember>. Staff members must be sorted primarily by surname and secondarily by staff ID.
2. Implement the compareTo() method correctly. It must perform alphabetical comparison and must not produce incorrect ordering when two staff members have the same surname.
3. In WestminsterHealthCentreManager, modify the printStaffMemberList() method so that it sorts the list using Collections.sort() before printing.

6 marks

5 marks

4 marks

4. Add a new method called `printActiveStaffOnly()` in `WestminsterHealthCentreManager`. This method must display only staff members whose active status is true.

4 marks

5. Add another method called `printStaffByRole()` that asks the user to enter a role such as Doctor, Receptionist, LaboratoryTechnician, or Pharmacist, and prints only matching staff members.

4 marks

6. Update the console menu so that both new listing options are accessible to the user.

2 marks

TASK 04 – Modify the Staff Table GUI

10 marks

Modify `StaffTableModel.java` so that the GUI table becomes more useful for operational decision-making.

1. Add two additional columns to the GUI table:

- Staff ID
- Status

4 marks

2. Modify `getValueAt()` so that the correct staff ID is displayed for each staff member.

2 marks

3. Modify `getValueAt()` so that the status column displays Active or Inactive based on the staff member's current status.

2 marks

4. Ensure that the existing name, surname, and role columns continue to work correctly after the modification.

2 marks

TASK 05 – Add Workload Statistics to the GUI

15 marks

The health centre manager wants to identify operational workload pressure through the GUI.

Modify StaffTableGUI.java by adding an event handler to the existing Statistics button.

When the button is clicked, the GUI must display:

- the doctor with the highest number of consultations per week;
- the number of consultations for that doctor;
- the total number of active staff members.

Your implementation must include the following:

1. Add a proper event handler to the Statistics button using an ActionListener.

4 marks

2. Correctly loop through the staff list and identify only Doctor objects.

3 marks

3. Correctly find the doctor with the highest number of consultations per week.

3 marks

4. Correctly count the number of active staff members.

2 marks

5. Display the results clearly in the GUI using a message dialog or another suitable Swing component.

2 marks

6. Handle the case where no doctors exist in the list without crashing the program.

1 mark

TASK 06 – Create JUnit Tests for Staff Status and Receptionist Desk Number

10 marks

Create a JUnit test class using the JUnit framework integrated into NetBeans.

The test class must verify the following:

1. A newly created Receptionist object correctly stores the name and surname passed through the constructor.
2 marks
2. The setDeskNumber() and getDeskNumber() methods in the Receptionist class work correctly.
2 marks
3. A newly created staff member is active by default.
2 marks
4. The setActive(false) method correctly changes the status of a staff member to inactive.
2 marks
5. The test class must contain meaningful assertions. Empty auto-generated tests or tests without assertions will not receive marks.
2 marks